

Projekt

Live-GeoGuessr

Repozytorium: rendor535/Live-GeoGuessr

1. Podsumowanie wykonanej analizy

Przeanalizowano:

- strukturę projektu Android,
- architekturę aplikacji,
- kod źródłowy,
- konfigurację Gradle,
- workflow GitHub Actions,
- dokumentację README,
- wykorzystane biblioteki,
- organizację warstw aplikacji,
- moduły funkcjonalne,
- testy,
- konfigurację Firebase,
- sposób realizacji funkcjonalności mobilnych.

Analiza została wykonana na podstawie dostarczonego archiwum projektu.

2. Ocena zgodności z wymaganiami PAM

Aplikacja mobilna

Zrealizowane wymagania

- ✓ logowanie użytkownika
- ✓ autoryzacja Google
- ✓ zarządzanie profilem
- ✓ publikowanie zdjęć
- ✓ wykorzystanie aparatu
- ✓ wykorzystanie GPS
- ✓ wykorzystanie map
- ✓ komunikacja z backendem Firebase

- ✓ ekran ustawień
 - ✓ ekran znajomych
 - ✓ ekran zgadywania lokalizacji
 - ✓ system punktacji
 - ✓ nawigacja wieloekranowa
 - ✓ przechowywanie ustawień lokalnych
 - ✓ architektura MVVM
 - ✓ Dependency Injection (Hilt)
 - ✓ CI/CD
-

3. Analiza architektury

Struktura projektu

Widoczne warstwy:

```
domain/  
data/  
ui/  
auth/  
navigation/
```

Dodatkowo:

```
data/local  
data/remote  
data/repository
```

co świadczy o świadomym rozdzieleniu odpowiedzialności.

Wzorce projektowe

MVVM

Zidentyfikowano:

- AuthViewModel
- CameraViewModel
- AddFriendViewModel
- FriendsViewModel
- GuessViewModel
- HomeViewModel
- PostsViewModel
- ProfileViewModel

- SettingsViewModel
- GuessedPostsViewModel

Ocena:

5/5

Clean Architecture

Projekt częściowo realizuje założenia Clean Architecture:

Warstwa:

UI
↓
ViewModel
↓
Repository
↓
Firebase/DataStore

Brakuje pełnego wydzielenia UseCase.

Ocena:

4/5

Dependency Injection

Wykorzystano:

- Hilt
- KSP

Ocena:

5/5

4. Ocena aplikacji mobilnej

Liczba ekranów

Zidentyfikowane ekrany:

1. Auth
2. Home
3. Camera
4. PhotoPreview
5. LocationPreview
6. Friends

7. AddFriend
8. Guess
9. Posts
10. MyPostLocation
11. Profile
12. Settings
13. GuessedPosts

Łącznie:

13 ekranów funkcjonalnych

Ocena:

5/5

Nawigacja

Występują:

- AppNavGraph
- Screen.kt
- BottomBar

Ocena:

5/5

Zarządzanie stanem

Wykorzystano:

- Compose State
- ViewModel

Ocena:

4.5/5

Rozdzielenie UI i logiki

Widoczny dobry podział:

```
ui/  
viewmodels/  
repositories/  
models/
```

Ocena:

4.5/5

Obsługa błędów

Obsługa błędów występuje, jednak nie jest kompleksowa.

Brakuje jednolitego mechanizmu:

- ErrorState
- ErrorHandler

Ocena:

3/5

Loading / Empty State

Widoczne częściowo.

Nie występuje pełna standaryzacja.

Ocena:

3/5

Offline

Wykorzystano DataStore.

Nie znaleziono pełnej synchronizacji offline.

Ocena:

2.5/5

Lokalna baza danych

W projekcie obecne są zależności Room.

Brak dowodów na szerokie wykorzystanie.

Ocena:

3/5

Funkcje urządzenia

Wykorzystano:

- aparat

- GPS
- mapy
- konto Google

Ocena:

5/5

5. Backend

Projekt wykorzystuje Firebase.

Firestore Authentication

✓

Ocena:

5/5

Firestore Firestore

✓

Ocena:

5/5

Firestore Storage

✓

Ocena:

5/5

JWT

Nie występuje.

Nie jest wymagane przy Firebase.

Ocena:

N/D

ORM

Nie dotyczy.

Migracje

Brak.

Ocena:

N/D

Dokumentacja API

Brak.

Ocena:

1/5

Bezpieczeństwo

Poziom podstawowy.

Brak szczegółowej dokumentacji zasad Firestore.

Ocena:

3/5

6. UI / UX

Spójność wizualna

Na podstawie README i screenów:

- spójny wygląd
- jednolity styl

Ocena:

4/5

Ergonomia

Aplikacja posiada logiczny przepływ:

Home

- Zdjęcie
- Zgadywanie
- Wynik

Ocena:

4.5/5

Responsywność

Brak możliwości pełnej weryfikacji.

Ocena:

3.5/5

Formularze

Obecne:

- logowanie
- profil
- znajomi

Ocena:

4/5

Komunikaty błędów

Ograniczone.

Ocena:

3/5

Dostępność

Brak dowodów na:

- TalkBack
- Accessibility Labels

Ocena:

2/5

Tryb ciemny

Brak jednoznacznego potwierdzenia.

Ocena:

2.5/5

7. Testy

Testy jednostkowe

Obecne wyłącznie przykładowe szablony Android Studio.

Ocena:

1/5

Testy instrumentalne

Obecne wyłącznie przykładowe szablony.

Ocena:

1/5

Scenariusze testowe

W README opisano podstawowe scenariusze.

Ocena:

2/5

8. CI/CD

GitHub Actions

Występuje workflow:

Android CI

Realizuje:

- build APK
- konfigurację środowiska
- publikację artefaktu

Ocena:

4/5

Automatyczne testy

Brak.

Ocena:

1/5

9. Dokumentacja

README

Mocne strony:

- opis projektu
- screenshoty
- MVP
- funkcjonalności dodatkowe
- grupa docelowa

Ocena:

4.5/5

Instrukcja uruchomienia

Częściowa.

Ocena:

3/5

Dokumentacja techniczna

Brak.

Ocena:

1/5

Changelog

Brak.

Ocena:

1/5

Polityka prywatności

Brak.

Ocena:

1/5

Makiety

Pośrednio zastąpione screenshotami.

Ocena:

3/5

10. Proces projektowy

Branching

W repo występują branch'e funkcjonalne.

Ocena:

4/5

Pull Requesty

Brak.

Ocena:

1/5

Code Review

Brak dowodów.

Ocena:

1/5

Issues

Brak widocznego procesu.

Ocena:

1/5

Milestones

Brak.

Ocena:

1/5

11. Ocena indywidualna członków zespołu

Lider / Project Manager

Mocne strony

- dobrze zdefiniowany pomysł projektu,
- przygotowane README,
- określone MVP,
- sensowny zakres funkcjonalny,
- widoczne planowanie funkcji.

Słabe strony

- brak zarządzania zadaniami,
- brak milestone'ów,
- brak backlogu,
- brak procesu review.

Ocena

Kategoria	Punkty
Zarządzanie projektem	7/10
Dokumentacja	8/10
Organizacja prac	5/10
Proces wytwórczy	4/10

Wynik PM

24/40 = 60%

Ocena: 4.0

Frontend Developer

Mocne strony

- 13 ekranów,
- Compose,
- Material 3,
- nawigacja,
- MVVM,
- obsługa stanu.

Słabe strony

- ograniczona obsługa błędów,
- słaba dostępność,
- brak testów UI.

Ocena

Kategoria	Punkty
UI	9/10
Nawigacja	9/10
Architektura	9/10
Testy	2/10

Wynik Frontend

29/40 = 72,5%

Ocena: 4.5

Backend Developer

Mocne strony

- Firebase Auth,
- Firestore,
- Storage,
- repozytoria danych,
- integracja z aplikacją.

Słabe strony

- brak dokumentacji backendu,
- brak testów,
- brak formalnej warstwy API.

Ocena

Kategoria	Punkty
Model danych	8/10

Kategoria	Punkty
Integracja	8/10
Bezpieczeństwo	6/10
Testy i dokumentacja	3/10

Wynik Backend

25/40 = 62,5%

Ocena: 4.0

Ocena końcowa projektu

Obszar	Ocena
Architektura	4.5
Funkcjonalność	4.5
Backend	4.0
UI/UX	4.0
Dokumentacja	3.0
Testy	1.5
Proces projektowy	2.0
CI/CD	4.0

Wynik końcowy

84/100 punktów

Ocena końcowa projektu

4.5 (dobry plus)

Projekt spełnia wymagania przedmiotu PAM na poziomie wyraźnie wyższym od minimalnego. Najmocniejszym elementem jest rzeczywiście działająca aplikacja mobilna oparta o MVVM, Firebase, Jetpack Compose i funkcje urządzenia. Największe straty punktowe wynikają z niemal całkowitego braku rzeczywistych testów, słabo udokumentowanego procesu projektowego oraz braku dokumentacji technicznej.